Kafka Architecture: Zero-Copy & Linux sendfile() Syscall

How Apache Kafka achieves multi-gigabit throughput by bypassing user-space buffer copies with the Linux kernel sendfile() system call and DMA transfer.

1. Traditional File Transfer Bottleneck

Standard web servers execute 4 context switches and 4 buffer copies to serve a file: Disk ➔ OS Page Cache ➔ User-space Buffer ➔ Socket Buffer ➔ NIC DMA.

```
read(fd, buf, len) # 2 copies + 2 context switches
write(sock, buf, len) # 2 copies + 2 context switches
```

2. Linux sendfile() Zero-Copy Magic

Kafka requests the Linux kernel to transfer bytes directly between OS page cache and the network interface socket buffer, eliminating user-space memory copies.

```
sendfile(socket_fd, file_fd, NULL, count);
```

- Zero CPU cycle waste copying bytes
- Zero user-space memory allocation

3. DMA Hardware Transfer

Direct Memory Access (DMA) engines copy data straight from OS page cache into the NIC transmission ring without CPU intervention.

```
Disk ➔ OS Page Cache ➔ [DMA Copy] ➔ NIC Buffer ➔ Network
```

4. Benchmark & Scaling Advantage

By leveraging OS page cache directly, Kafka nodes can saturate 10Gbps and 40Gbps network cards with less than 15% CPU load:

- Sustained 1,000,000+ messages/sec per broker
- OS automatically manages RAM via dirty page flushes